ОПИСАНИЕ ПРОЦЕССА СОЗДАНИЯ ПРОТОТИПА

Дека — цифровой деканат будущего

Как мы собрали работающий личный кабинет студента вместе с ИИ-ассистентами: расписание с переносами, оценки, заявки на справки и ИИ-помощник — в одном окне, с телефона от 360 px. Внутри — инструменты, промпты, ошибки и проверки.

Конкурс «Цифровой деканат будущего»

Команда «Дека» · ноябрь 2026

Прототип не является официальным сервисом вуза. Все данные вымышленные.

Содержание

1. Проблема и решение 2

Почему нейтральный вуз и тёплый стиль . . . 2

2. Что получилось: обзор прототипа 3

Кабинет деканата — вторая сторона системы . . . 5

ИИ-ассистент «Дека» . . . 6

3. Как шла работа с ИИ-ассистентами 6

Ключевые приёмы постановки задач . . . 7

4. Ошибки, которые ИИ допустил, и как мы их ловили 8

5. Вход, данные и защита — простыми словами 9

Пароли и вход . . . 9

Сессия . . . 9

Каждый видит только своё . . . 9

6. Этапы, проделанные вручную 10

7. Тестовые аккаунты и как проверять прототип 10

Чек-лист проверки для жюри . . . 11

1. Проблема и решение

Учебная жизнь студента сегодня разбросана по нескольким не связанным между собой местам: расписание живёт на сайте вуза, оценки — в отдельной системе, справки оформляются через очередь в деканате, а объявления приходят в чаты и на почту. С телефона нужное найти особенно трудно: сайты вузов обычно рассчитаны на экран компьютера. Мы решили показать, каким может быть одно место, где всё это собрано и устроено так, как удобно современному студенту.

Ядро прототипа — обязательные разделы конкурса: вход и профиль, расписание, успеваемость, заявки и справки, уведомления. Поверх ядра мы добавили три идеи, которые, на наш взгляд, меняют привычный опыт общения с деканатом: ИИ-ассистент, который отвечает на вопросы по личным данным студента; электронный студенческий с QR-кодом вместо пластикового бейджа; экран учебного прогресса с наглядной картиной семестра. Ключевой принцип — «нужное за пару касаний»: любое действие студента занимает не больше трёх экранов.

Раздел	Задача студента	Как устроено в прототипе
Вход и профиль	Быстро войти, видеть свои данные	Логин и пароль, сессия в защищённой cookie; ФИО, факультет, направление, курс, группа, зачётка — на одном экране
Расписание	Знать, что сегодня и на неделе	Сегодня и неделя с переключением, аудитория и преподаватель, переносы и отмены подсвечены отдельным блоком «Изменения»
Успеваемость	Понять, «как у меня дела»	Балльно-рейтинговая система: баллы по модулям, прогноз оценки по каждой дисциплине, место в группе, расписание сессии с обратным отсчётом
Заявки и справки	Получить документ без очереди	Каталог из шести типов, подача в два касания, статус-трекер как у доставки: Принята → В работе → Готова → Выдана, QR-код выдачи
Уведомления	Не пропустить важное	Статусы заявок, изменения расписания, дедлайны и объявления деканата; непрочитанные подсвечены, есть отметка «прочитать все»
ИИ-ассистент «Дека»	Спросить вместо того, чтобы искать	Чат отвечает по личным данным: что сегодня, когда сессия, где моя справка; использует только данные этого студента

Таблица 1. Обязательное ядро и дополнительные функции

Почему нейтральный вуз и тёплый стиль

Мы сознательно сделали кабинет универсальным: нейтральный «Университет» без привязки к конкретному вузу позволяет показать концепт любому жюри и избежать использования официальной символики. Логотип «Дека» — собственный: стилизованная буква «Д» с искрой-акцентом,

отсылающая к ИИ-ассистенту. Визуальный стиль — тёплый и дружелюбный: кремовый фон, бирюзово-изумрудный акцент, мягкие скругления, понятные подписи без канцелярита. Такой язык interface-дизайна ближе к привычным студентам приложениям (Telegram, Duolingo), чем к «порталу успеваемости».

2. Что получилось: обзор прототипа

Прототип открывается по ссылке в браузере и полностью проходится мышкой и пальцем: вход, расписание, оценки, подача заявки, ответы ассистента, работа деканата. Все сценарии работают на экране шириной 360 px — оттуда и сделана большая часть скриншотов ниже.

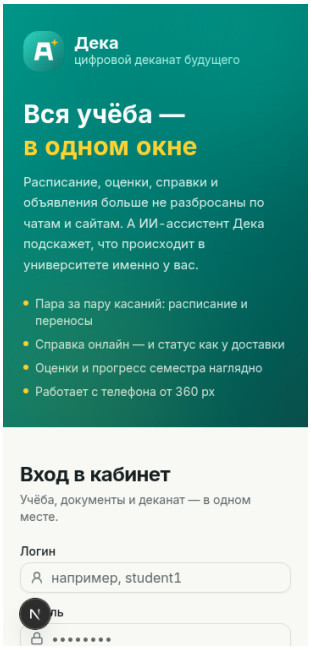


Рис. 1. Экран входа: тестовые аккаунты жюри в одном касании

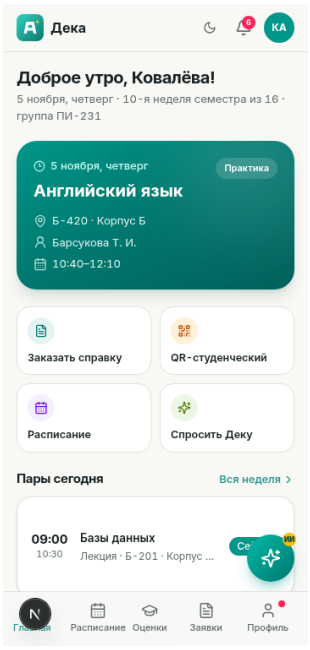


Рис. 2. Главная: ближайшая пара, быстрые действия, дедлайны

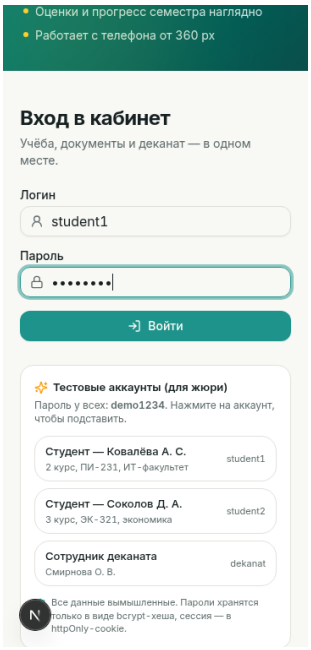


Рис. 3. Расписание: неделя, перенос лекции и отмена

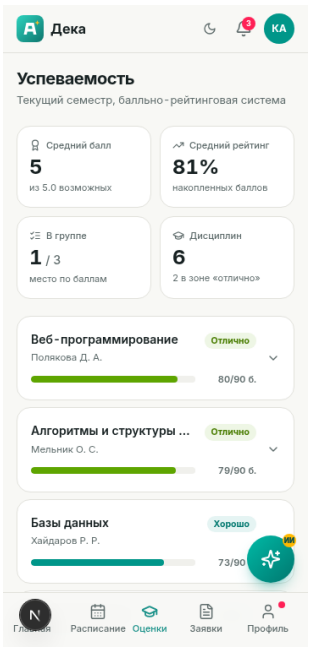


Рис. 4. Успеваемость: БРС, прогноз оценок, место в группе

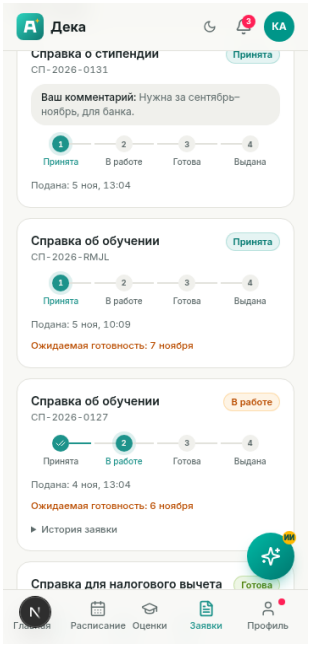


Рис. 5. Заявки: статус-трекер и ожидаемая готовность

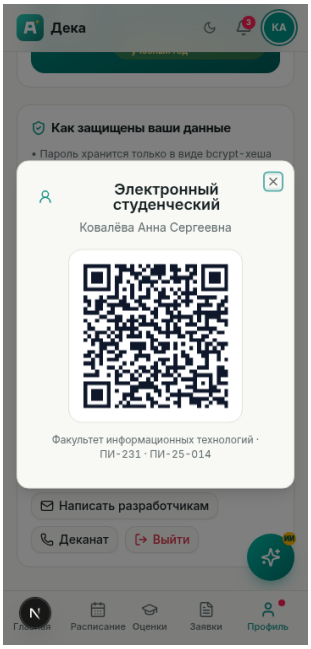


Рис. 6. Электронный студенческий с QR-кодом

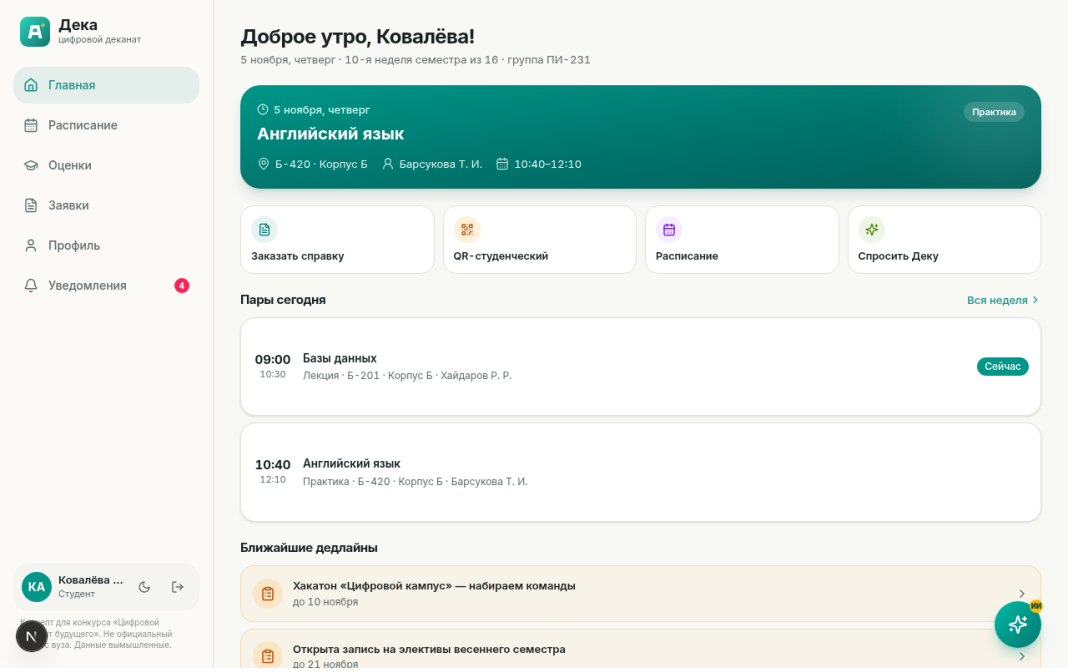


Рис. 7. Десктопная версия: та же главная с боковой навигацией

Кабинет деканата — вторая сторона системы

Чтобы показать полный цикл, мы сделали и кабинет сотрудника деканата: очередь заявок с фильтрами, публикация объявлений для всех или отдельной группы, переносы расписания (студенты получают уведомление сразу) и список студентов с поиском. Жюри может выйти студентом, подать заявку, затем войти сотрудником, взять её в работу — и увидеть у студента уведомление о новом статусе.

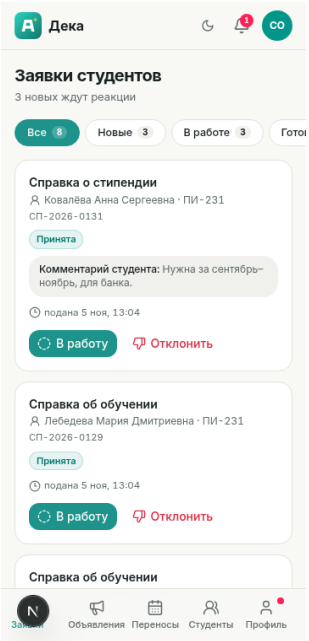


Рис. 8. Кабинет деканата: очередь заявок со счётчиками

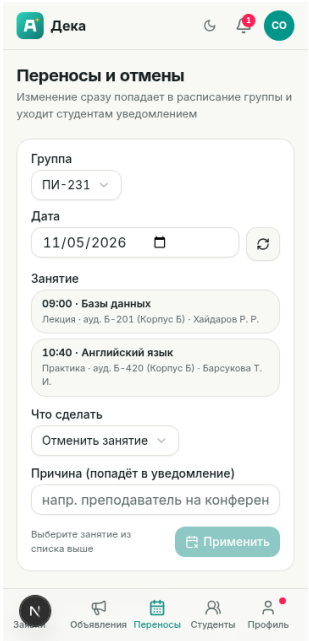


Рис. 9. Переносы: изменение уходит студентам уведомлением

ИИ-ассистент «Дека»

Ассистент отвечает на вопросы студента, опираясь только на его данные: расписание его группы, его оценки и заявки, объявления, которые ему видны. На вопрос «Когда начинается сессия?» Дека отвечает конкретно — с датой первого зачёта. Если данных не хватает, ассистент честно говорит об этом и подсказывает раздел кабинета. Ключи языковой модели лежат только на сервере и в браузер не попадают.

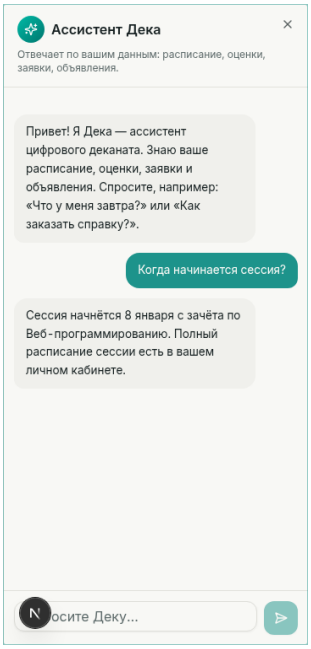


Рис. 10. Чат с ассистентом: ответ по реальным данным

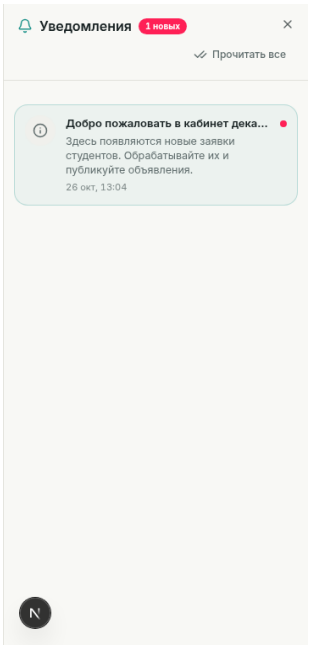


Рис. 11. Уведомления: непрочитанные выделены цветом

3. Как шла работа с ИИ-ассистентами

Прототип написан в парадигме vibe-coding: код целиком написан ИИ-агентом, человек ставил задачи, выбирал решения и проверял результат. Роль человека при этом не «заказчик», а редактор и тестировщик: каждый экран проходился руками, каждая ошибка воспроизводилась и отправлялась агенту на исправление вместе с диагнозом. Ниже — этапы, инструменты и примеры реальных постановок задач.

Этап	Чем делали	Результат этапа
1. Разбор брифа	ИИ-агент (Z.ai, модель GLM)	План работ на 10 шагов: БД, аутентификация, API, интерфейс, проверки, деплой
2. Уточнение требований	Человек + агент	Шесть решений: нейтральный вуз, тёплый стиль, ИИ-ассистент + QR + прогресс, кабинет деканата, инструкция деплоя, PDF-отчёт
3. Данные	Агент (Prisma, SQLite)	Схема из 11 таблиц и скрипт наполнения: 6 студентов, расписание двух групп, 77 оценок, 7 заявок с историей, 7 объявлений
4. Безопасность	Агент (bcrypt, jose JWT)	Хеши паролей, сессия в httpOnly-cookie, проверка сессии и роли в каждом API-роуте, rate-limit входа

Этап	Чем делали	Результат этапа
5. Интерфейс	Агент (Next.js 16, Tailwind 4, shadcn/ui)	SPA-кабинет: 5 разделов студента, 5 разделов деканата, мобильная навигация, тёмная тема, пустые состояния и скелетоны загрузки
6. ИИ внутри продукта	z-ai-web-dev-sdk (сервер)	Ассистент «Дека»: контекст из базы, системный промпт, запасной режим без LLM
7. Проверка	agent-browser + ESLint + логи	Все сценарии пройдены в браузере на 360 px и 1280 px; найденные дефекты исправлены и перепроверены
8. Доставка	Агент (Docker, pypdf)	Dockerfile и docker-compose, инструкция DEPLOY.md, README, этот PDF-отчёт

Таблица 2. Этапы разработки и роль инструментов

Ключевые приёмы постановки задач

Первый и самый важный промпт — это сам конкурсный бриф: он уже содержит требования, критерии оценки и ограничения. Мы отдали его агенту целиком и попросили сначала составить план, не переходя к коду. Дальше работали короткими итерациями: одна итерация — одна проверяемая функция. Типичная формулировка этапа выглядела так:

> *«Сделай раздел “Заявки” студента: каталог типов справок из БД, диалог подачи (тип, комментарий, валидация), список своих заявок со статус-трекером из 4 шагов и историей. API: GET /api/requests (только свои), POST /api/requests с генерацией номера СП-/ЗВ-. Проверь: подача новой заявки студент1 появляется в списке и в очереди деканата». — этап 5, постановка раздела заявок*

Когда найдена ошибка, мы отправляли агенту не «исправь баг», а symptom + диагноз: что делали, что ожидали, что получили, что показал лог. Это заметно сокращало число итераций. Например:

> *«POST /api/auth/login возвращает 400 “Введите логин и пароль”, хотя поля заполнены. Судя по network-логу браузера, fetch не отправлялся до requestSubmit(). Проверь валидацию в login-роуте и тип контролируемых инпутов». — этап 7, отладка*

Отдельный приём — «проверь сам»: агенту давали задачу прогнать сценарий через agent-browser (открыть, войти, кликнуть, снять скриншот, прочитать ошибки консоли) и только потом отчитаться. Такой замкнутый цикл «сделал — проверил — исправил» занял большую часть времени этапа качества.

11 таблиц
в схеме базы данных

16 API-роутов
с проверкой сессии и роли

360 px
минимальная ширина сценариев

3 роли
в тестовых аккаунтах жюри

4. Ошибки, которые ИИ допустил, и как мы их ловили

ИИ-агент ошибался предсказуемо: путал ограничения платформы, оставлял мусор от удалённых файлов и не проверял граничные случаи арифметики дат. Ниже четыре реальных случая из протокола разработки — с симптомом, диагнозом и исправлением. Все они были найдены проверками, а не чтением кода: сборкой, логами и браузерным прогоном.

Симптом	Диагноз	Исправление и проверка
При входе с «просроченной» сессией — белая страница и ошибка 500	Агент вызывал изменение cookie прямо при отрисовке страницы; Next.js разрешает менять cookie только в серверных действиях и API	Убрали сброс cookie из рендера: страница показывает экран входа, cookie перезаписывается при следующем входе. Проверено: вход после пересева БД
Сервер отвечает 500 на все страницы, в логе — ошибка парсинга CSS	В кэше Tailwind остался класс из удалённого файла-макета; файл удалён, а сгенерированный CSS остался	Форсировали пересборку стилей и вычистили классы-дубли. Проверено: сборка чистая, страница открывается
На главной показывалась «62-я неделя семестра из 16»	Неделя считалась от 1 сентября года поступления, а не текущего учебного года	Функция теперь считает от 1 сентября текущего учебного года с ограничением 16 неделями. Проверено: 5 ноября — 10-я неделя
Заголовок панели уведомлений на 360 px обрезал кнопку «Прочитать все»	Строка заголовка не переносилась: бейдж счётчика и кнопка не помещались	Перенос строки в заголовке (flex-wrap) и сокращённые подписи. Проверено: скриншот «до/после» на 360 px (рис. 12)

Таблица 3. Реальные дефекты ИИ-кода и их исправление

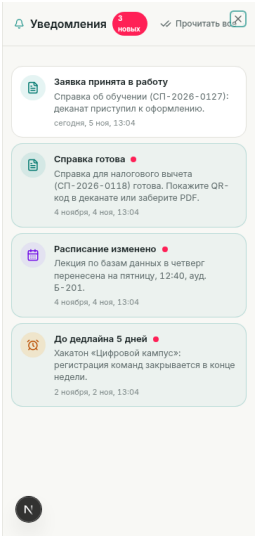


Рис. 12а. До: кнопка «Прочитать все» обрезана

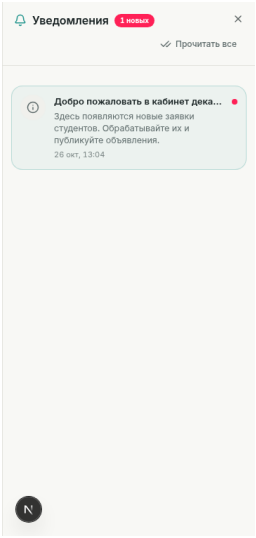


Рис. 12б. После: заголовок переносится, всё видно

Общий вывод: агент пишет быстро, но «из головы» — без запуска. Поэтому в наш цикл входили обязательные внешние проверки: ESLint на правила React, чтение лога dev-сервера после каждой правки, браузерный прогон agent-browser с просмотром консоли и сети, а после финала — сквозной

сценарий обеими ролями. Именно проверки, а не повторные промпты, дали основную прибавку качества.

5. Вход, данные и защита — простыми словами

Мы заложили безопасность в основу прототипа, а не «доделали потом». Ниже — как это устроено без технического жаргона, и что из этого реально проверяется жюри за пару минут.

Пароли и вход

- Пароль никогда не хранится в открытом виде: в базе лежит только его «отпечаток» (bcrypt-хеш). Даже человек с доступом к базе не сможет узнать пароль.
- При входе сервер сравнивает отпечатки. Если логина нет, сервер всё равно «прокручивает» сравнение с фиктивным хешем — чтобы нельзя было перебором узнать, какие логины существуют.
- Не больше 10 попыток входа в минуту с одного адреса: защита от перебора паролей.

Сессия

- После входа сервер выдаёт подписанный «пропуск» (JWT) и прячет его в cookie, которую не может читать JavaScript браузера — украть её скриптом нельзя.
- Пропуск подписан секретом, который лежит только в переменных окружения сервера. В коде и в репозитории его нет: в репозитории есть только шаблон .env.example.

Каждый видит только своё

- Каждый запрос к серверу заново проверяет пропуск и роль. Студенческие разделы отвечают только данными того, чей пропуск пришёл; чужой id в адрес подставить нельзя — сервер фильтрует по пропуску, а не по адресу.
- Разделы деканата отвечают только пропуску сотрудника: студент получает отказ «403». Проверено автоматически: запрос студента к очереди деканата возвращает отказ, запрос без входа — «требуется вход».
- Ключи ИИ-сервиса вызываются только на сервере и в браузер не попадают.

Угроза	Защита в прототипе
Подбор пароля	bcrypt-хеш + фиктивное сравнение для несуществующих логинов + лимит 10 попыток в минуту
Кража сессии скриптом	JWT в httpOnly-cookie (SameSite=Lax, Secure в продакшене), секрет в переменных окружения
Чужие заявки и оценки по подставленному адресу	Фильтрация по userId из сессии в каждом запросе; роли проверяются заново на каждом API-роуте
Утечка ключа ИИ	SDK вызывается только на сервере; в клиентский код ключ не попадает

Угроза	Защита в прототипе
Реальные персональные данные	Все ФИО, группы, оценки и номера вымышлены; прототип не подключён к системам вуза

Таблица 4. Угрозы и меры защиты

6. Этапы, проделанные вручную

Хотя код писали ИИ-ассистенты, без человека прототип не собрался бы. Вот что сделали мы сами — и что, по нашему опыту, автоматизировать не стоит.

- **Постановка задачи.** Перевели конкурсный бриф в план из 10 шагов и приоритеты: сначала работающее ядро, затем фишки, затем полировка.
- **Дизайн-решения.** Выбрали нейтральный вуз, тёплый стиль, состав дополнительных функций (ассистент, QR-студенческий, прогресс), необходимость кабинета деканата — это осознанные решения о продукте, а не про код.
- **Проверка сценариев.** Каждый экран проходили руками как студент и как сотрудник деканата, в том числе на телефоне шириной 360 px; фиксировали, где «что-то не так» — и формулировали это агенту.
- **Правки смысла.** Например, решение показывать дедлайны прямо на главной и называть статусы «человеческими» словами («Готова к выдаче», а не «READY») — это было прочитано и решено людьми.
- **Инфраструктура доставки.** Выбор хостинга для доступа из России без VPN (VPS с Docker как основной путь), решение о структуре репозитория и о том, какие файлы не должны в него попадать (секреты, база).

7. Тестовые аккаунты и как проверять прототип

Для жюри подготовлены три аккаунта с разными данными, чтобы было видно разделение: у двух студентов разные факультеты, группы, расписания, оценки и заявки. Пароль у всех один — demo1234.

Логин	Пароль	Кто это	Что посмотреть
student1	demo1234	Ковалёва А. С., 2 курс, группа ПИ-231, ИТ-факультет	Главная с ближайшей парой; 4 заявки в разных статусах; QR-студенческий; ассистент
student2	demo1234	Соколов Д. А., 3 курс, группа ЭК-321, экономика	Другое расписание и оценки — данные не пересекаются с student1
dekanat	demo1234	Смирнова О. В., сотрудник деканата	Очередь заявок (взять в работу), публикация объявления, перенос пары — затем войти студентом и увидеть уведомление

Таблица 5. Тестовые аккаунты (все данные вымышленные)

Чек-лист проверки для жюри

- Открыть ссылку с телефона (или в узком окне): вход, главная, расписание, оценки, подача заявки — всё доступно без горизонтальной прокрутки.
- Войти student1 и student2 по очереди: расписания, оценки и заявки различаются — данные разделены.
- Подать заявку студентом, войти деканатом, сменить статус; вернуться студентом — в уведомлениях появился новый статус.
- Войти деканатом, перенести пару у группы ПИ-231 — у студентов группы появится уведомление, а в расписании изменится день.
- Спросить ассистента: «Что у меня завтра?» — ответ строится по данным конкретного аккаунта.
- На экране входа и в подвале кабинета есть отметка, что это концепт для конкурса, а не официальный сервис вуза.

Исходный код — в репозитории команды (ссылка в комментарии к работе), там же инструкция DEPLOY.md: прототип поднимается одной командой docker compose up на любой VPS и работает без VPN из России. Ссылка на работающий прототип и пароли аккаунтов передаются в комментарии к работе. Все данные прототипа вымышленные; пароли тестовых аккаунтов существуют только внутри прототипа.